

JAVA OBJECTS:

when we creating object of a class we actually making an instance of the class

→ So object is something that exist.

Properties of OBJECT:

- State of object
- Identity of object.
- Behaviour of object.

"new" keyword in Java

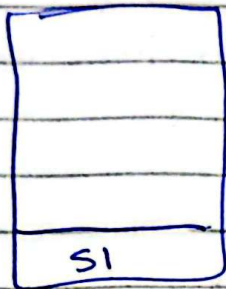
for example we have a class student and it have different properties like roll no, name, age etc.

So when in java we do

```
Student S1; // we are declaring the  
// reference variable
```

So basically this is the reference to an object of type student.

So in stack memory it is like



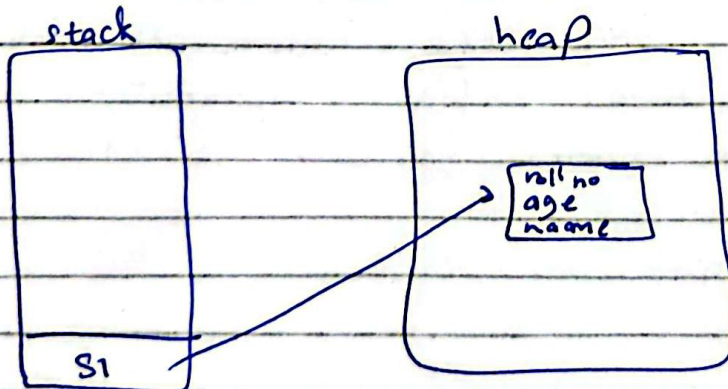
but it is null because it is not pointing to something

then if we do this:

```
s1 = new Student();
```

what it will do is dynamically allocates the memory and returns it to s1.

So now in the stack memory



The object is pointing to memory in heap

Dynamic memory allocation:

when we do

Student S1 = new Student();
compile time run time

So the memory is allocated when program is executed.

→ This is for constructor.

Constructor:

~~It~~ A constructor is a function that ~~ans.~~ runs when you create a new object, it ~~allocates~~ creates and initializes the object.

So basically

S1 = new Student("Saaim", 23);

→ It will bind these arguments with the object.

it don't have any return type.

"this" Keyword in Java:

It is reference to the current object. The object whose method or constructor is being called.

Whenever we are trying to access the particular item of the class through its object, that's when it is used.

Constructor Overloading :

two to three Constructors like

```
Student () {
```

```
    this.rno = 13;
```

```
    this.name = "Saam";
```

```
}
```

OR with arguments

```
Student (int rno, String name) {
```

```
    this.rno = rno;
```

```
    this.name = name;
```

```
}
```

We can a constructor from another constructor as well.

what we do is

```
Student () {
```

```
    this ( 13, "saaim", 44.2);
```

```
}
```

```
Student (int rollno, String name, float marks) {
```

```
    this.rollno = rollno;
```

```
    this.name = name;
```

```
    this.marks = marks
```

```
}
```

) Internally it is like : new Student(13,-);

Why we don't use "new"

Key word for primitive datatypes?

Because in Java primitive datatypes are not treated as objects.